

Given your deck's own timeline — features due "by 1 Oct" — that leaves you roughly a day. So before the stack: **cut scope now, not later**. The email-triage-and-draft loop (fetch → classify → draft → approve → send) is a complete, demoable agent on its own. Calendar slot-finding and task extraction are real features, but if time gets tight, a deep, reliable version of feature 1 beats a shaky version of all three.

Stack, by layer:

- **Frontend** — React (Vite) + Tailwind. For live updates to the approval queue, either poll every few seconds with plain `fetch` , or use Supabase's realtime subscriptions if you go that route (see below) — less code than building your own polling logic.
- **Backend** — Node.js + Express. Key packages: `googleapis` (official Gmail + Calendar v3 client), `google-auth-library` (OAuth2 flow + token refresh), `zod` (validate the LLM's structured output before you act on it), `dotenv` .
- **LLM/agent layer** — Whichever of DeepSeek / Gemini Flash / Groq you land on, they're all interchangeable here: DeepSeek and Groq are OpenAI-compatible (use the `openai` npm package, just point `baseURL` at theirs), Gemini uses `@google/generative-ai` . All three support function/tool calling in roughly the same shape, so swapping later is a config change, not a rewrite.
- **Data** — Supabase (Postgres, free tier, realtime, zero server setup) is the fast option — two tables: `actions` (pending/approved/rejected) and `tasks` . `better-sqlite3` is the fallback if you'd rather not deal with another signup.
- **Hosting** — Railway or Render for the backend (you need a long-running process, not pure serverless, since it's polling/syncing). Vercel for the frontend.

Build order:If you split this across your team (Agent/Backend, Integrations, Frontend), steps 2+4+5 are one track, 1+3 another, and 7 the third — they can run in parallel rather than sequentially. Want me to scaffold the actual starter code (backend + the LLM tool-schema) so you're not writing boilerplate from scratch?